

Desafio Técnico — Desenvolvedor .NET Júnior

O objetivo deste desafio é desenvolver uma pequena API para gerenciamento de **produtos e pedidos**.

Não é necessário desenvolver interface gráfica. Toda a aplicação poderá ser testada através do Swagger.

Tecnologias

Utilize:

- .NET / ASP.NET Core Web API
- Entity Framework Core
- Banco de dados relacional de sua escolha
- Swagger

A aplicação deve utilizar `async/await` nas operações que envolvem acesso ao banco de dados.

Estrutura esperada

Não existe uma arquitetura obrigatória, mas esperamos uma separação mínima de responsabilidades.

Por exemplo:

- Controllers ou Endpoints
- Interfaces
- Services
- Models / Entities
- Acesso ao banco de dados
- Injeção de Dependência (DI)

Você pode organizar o projeto da maneira que considerar mais adequada.

Entidades

Produto

Um produto deve possuir pelo menos:

- Id
- Nome
- Categoria

- Preço
- Quantidade em estoque
- Data de cadastro

Pedido

Um pedido deve possuir pelo menos:

- Id
- Produto
- Quantidade
- Valor total
- Data do pedido

Funcionalidades

Produtos

Criar os endpoints necessários para:

- Cadastrar um produto
- Buscar um produto pelo Id
- Atualizar um produto
- Excluir um produto
- Listar produtos

Também deverá existir uma consulta que permita filtrar produtos por:

- Nome
- Categoria
- Faixa de preço

Não é necessário que todos os filtros sejam informados ao mesmo tempo.

Pedidos

Criar um endpoint para realizar um pedido.

Ao realizar um pedido:

1. O produto deve existir.
2. Deve existir estoque suficiente.
3. O estoque do produto deverá ser atualizado.
4. O valor total deverá ser calculado com base no preço e quantidade.

Também deverá existir um endpoint para listar os pedidos realizados.

Paginação

A listagem de produtos deve possuir paginação.

Exemplo:

```
GET /products?page=1&pageSize=20
```

A implementação e a estrutura da resposta ficam a seu critério.

Banco de dados

Utilize um banco relacional.

Pode ser utilizado, por exemplo:

- PostgreSQL
- SQL Server
- MySQL
- SQLite

O projeto deverá possuir alguma forma simples de criação/atualização da estrutura do banco, como migrations.

Docker

Como diferencial, você pode disponibilizar um `docker-compose` para subir o banco de dados utilizado pela aplicação.

Não é obrigatório colocar toda a aplicação dentro do Docker.

Tratamento de erros

A API deverá tratar pelo menos situações como:

- Produto não encontrado
- Estoque insuficiente
- Dados inválidos

A forma como esses erros serão retornados pela API fica a seu critério.

README

Inclua um pequeno README explicando:

- Como executar o projeto
- Qual banco foi utilizado
- Como criar/configurar o banco
- Decisões que você considerou importantes durante o desenvolvimento